

Underdetermined Systems

Plain-English introduction to the benchmark, its experimental structure, the numerical methods, and the reproducibility workflow

1. What this section contains

This folder contains the computational material used for the large underdetermined-system part of my dissertation. The aim of the material is not simply to provide code that produces a set of numbers. I have kept the Python benchmark and the plotting stage separate so that another researcher can see what was calculated, reproduce the numerical runs, and regenerate the figures and summary tables from the resulting CSV files.

The two Python files in this folder are `Underdetermined.py` and `Underdetermined_Plots.py`. The first performs the numerical benchmark for one experiment and one solver at a time. The second collects the completed results and produces the summaries and plots used for inspection. The other two PDFs explain how to run the files and which computational conditions need to be respected.

2. What is an underdetermined system?

A linear system can be written as $Ax = b$. Here A is the coefficient matrix, x is the vector of unknowns, and b is the right-hand side. If A has m rows and n columns, the system is

$$\begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \dots & \vdots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix}$$

A is $m \times n$
 x is $n \times 1$
 b is $m \times 1$

For the experiments in this dissertation, the systems are synthetic. A known reference vector, x_{true} , is generated first. The matrix A is then generated according to the chosen dimensions, sparsity, pattern, condition parameter and spectrum. The right-hand side is constructed from A and x_{true} and includes a controlled noise level.

Because x_{true} is known, the calculation can be assessed using both the residual and the relative error against that reference solution. In plain terms, the experiment asks how different numerical methods behave when a large problem has many more unknowns than equations, and when the structure of that problem is changed in a controlled way.

3. Why the experiment uses synthetic matrices

Using synthetic matrices makes it possible to change one property at a time while keeping the rest of the problem under control. This is important because a change in matrix size, sparsity, conditioning, regularisation, sparsity pattern or spectral distribution changes the numerical problem itself. The resulting runtime and convergence behaviour therefore cannot be interpreted independently of those settings.

The matrix is stored as a sparse matrix. This is essential at the dimensions used here. The benchmark is not intended to turn the large underdetermined matrices into dense NumPy arrays. The sparse representation is part of the computational design.

4. The six experimental studies

Study	Main quantity varied	Question being examined
Matrix Size	n , with m fixed	How does increasing the number of unknowns affect runtime, iterations and accuracy?

Study	Main quantity varied	Question being examined
Sparsity	s	How does increasing the proportion of zero entries affect the calculation?
Sparsity Pattern	random, banded, block	Does the arrangement of non-zero entries matter?
Condition Parameter	κ	How does increasing the intended conditioning difficulty affect convergence and cost?
Regularisation	λ	How does regularisation change solver behaviour?
Spectrum	linear, exponential, polynomial, clustered	How does the distribution of matrix scaling affect numerical behaviour?

5. Parameter values used in the dissertation

The following values are the experimental ranges stated in the dissertation for the large underdetermined systems. The current Python file should be treated as the implementation-level reference when checking the exact configuration.

Parameter	Underdetermined setting
Dimensions	m = 5000; n = 15000, 20000, 40000 for the matrix-size study
Condition parameter	10, 100, 1000, 10000, 20000
Sparsity	0.90, 0.95, 0.98, 0.99, 0.999
Regularisation	0, 1e-8, 1e-6, 1e-4, 1e-2, 1e-1, 1
Sparsity pattern	random, banded, block
Spectrum	linear, exponential, polynomial, clustered
Repeats	10 runs per configuration
Noise level	0.01
Initial random seed	42

6. The actual study configurations

The six studies are not full Cartesian products of every parameter listed above. For each study, the supplied code changes the intended parameter while fixing the other values. The configurations implemented in the benchmark are:

Study	m	n	Condition	Sparsity	Lambda	Pattern	Spectrum
Condition Number	5000	15000	10 to 20000	0.95	1	random	exponential
Sparsity	5000	15000	1000	0.90 to 0.999	1	random	exponential
Matrix Size	5000	15000, 20000, 40000	1000	0.95	1	random	exponential
Lambda	5000	15000	1000	0.95	0 to 1	random	exponential
Sparsity Pattern	5000	15000	1000	0.95	1	random, banded, block	exponential
Spectrum	5000	15000	1000	0.95	1	random	linear, exponential, polynomial, clustered

7. Solvers compared

The benchmark contains four solver options: Direct, CG, LSQR and LSMR. The dissertation's successful numerical comparisons for the large underdetermined systems are based on CG, LSQR and LSMR. Direct was also tested, but it did not complete successfully for these large underdetermined cases and is therefore not included in the reported numerical comparison tables.

Solver	Role in the experiment
Direct	Direct reference calculation. It explicitly forms the normal matrix $A^T A$ before sparse direct solution. It was attempted, but the large underdetermined cases did not complete successfully.
CG	Iterative conjugate-gradient solution of the regularised normal-equation system using a matrix-free operator. The large code does not explicitly build $A^T A$ for CG.
LSQR	Iterative least-squares method operating on the sparse matrix A without explicitly constructing $A^T A$.
LSMR	Iterative least-squares method operating on the sparse matrix A without explicitly constructing $A^T A$.

8. What is measured

For each successful run the benchmark records the solver name, experiment, matrix dimensions, condition parameter, sparsity, sparsity pattern, spectrum, lambda, run number, runtime, iteration count, residual and relative error.

Runtime measures the elapsed time spent in the solver calculation. Iteration count records the number of iterations reported by the iterative method. The residual is based on the norm of $Ax - b$. Relative error compares the computed solution with the known synthetic x_{true} . Results are then averaged over the repeated runs for the summary tables.

9. How the files fit together

Underdetermined.py is the numerical stage. It accepts an experiment and solver through command-line arguments, generates the synthetic problem, performs the requested repeated runs and writes checkpoint and final CSV files.

Underdetermined_Plots.py is the post-processing stage. It searches the underdetermined output tree for completed results_final.csv files, combines them, creates summary CSV files and writes the figures into a plots directory.

10. What reproducibility means here

For this part of the dissertation, reproducibility means preserving the mathematical experiment and the solver implementation. The researcher reproducing the work may have to change the CSF partition, CPU allocation or other resource directives because University HPC policy changes. Those are execution-environment choices. Changing the matrix dimensions, parameter values, random seed, tolerances or solver formulation is a change to the experiment and should not be made silently.

The companion Run Instructions document gives the practical sequence from obtaining the files and establishing a CSF session through to collecting the CSV files and generating the plots. The Requirements and Restrictions document records the resource requirements and the conditions that should remain fixed.